Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Dersin tarihi 18 Kasım 2020 Çarşamba, günün ikinci oturumu (normalde uygulama saati). Ders Zoom üzerinden yapıldı.
  - Geçen hafta yarım kalan dersin telafisi için uygulama saati konu anlatımına ayrıldı.
  - Kaydın başında bağlantı sorunu yüzünden yaklaşık 5 dakika gecikme var.
- Ayrık zincirleme (separate chaining)
  - Bağlı listeye başa ekleme
  - Hangi durumda kullanılmalı ($n > m$)
  - Karmaşıklık
  - C ile `struct node`, `struct hash`, `insertToHash`, `createNode`, `searchInHash`
- Evrensel hashing (universal hashing)
- Yük faktörü (load factor) ve ortalama sınama (probe) sayıları
  - Başarısız aramada (unsuccessful search) adım sayısı türetmesi
  - Başarılı ve başarısız arama formülleri (doğrusal sınama)
  - Ayrık zincirlemede başarılı ve başarısız arama
- Doğrusal sınama ile ikinci dereceden sınamanın karşılaştırması
- Vize kapsamı

Hocanın Özellikle Vurguladığı Kısımlar

- Ayrık zincirlemede yeni elemanı listenin sonuna değil, başına eklemek daha akıllıcadır.
  - Sona eklemek için listenin sonuna kadar gitmek gereksiz maliyettir.
  - Sona eklemek gerekirse liste sonunu gösteren bir `tail` işaretçisi tutulur.
- Ayrık zincirleme, eleman sayısı tablo boyutundan büyükken ($n > m$) tercih edilir. Açık adresleme $m > n$ iken tavsiye edilir.
  - $m > n$ iken ayrık zincirleme de kullanılabilir. Ama $n > m$ iken açık adresleme kullanılamaz, tablo dolar.
- Ayrık zincirlemede teorik en kötü durum $O(n)$'dir (bütün elemanlar aynı göze düşerse). Pratikte $O(1)$ kabul edilir.
  - Bütün elemanların aynı göze düşmesi için çok kötü bir hash fonksiyonu ya da çok kötü bir tablo boyutu seçilmiş olmalıdır.
- Yük faktörünün genelde $\alpha \approx 0.5$ civarında olması istenir.
- Tablo doldukça başarısız arama çok pahalılaşır: %90 dolu tabloda doğrusal sınamada ortalama 50,5 sınama.
- İkinci dereceden sınama, aynı doluluktaki tabloda doğrusal sınamadan daha az sınama ile arama yapar.
- Örneklerde kolaylık için $m = 6$ seçildi. Gerçek uygulamada m asal seçilmelidir.
- Hash konusu vizeye dahildir. Bir sonraki ödev büyük ihtimalle hash ile ilgili olacak.
- Derse kâğıt üstünde bağlı olmak bir şey ifade etmez; derse katkı beklenir.

Kısa Tekrar Notları

- Ayrık zincirleme: Aynı adrese düşen elemanlar o gözün bağlı listesine eklenir. Liste sonu NULL gösterir.
- Örnek ($m = 6$): 13, 1, 19 anahtarlarının hepsi $k \bmod 6 = 1$ verir ve 1 numaralı gözün listesine eklenir.

- Ayrık zincirlemede ortalama sınama sayısı (hash hesabı dahil):
  - Başarılı arama: $1 + \alpha/2$
  - Başarısız arama: $1 + \alpha$ (yaklaşık α)
- Evrensel hashing: $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$. a, b, p rastgele seçilir, m sabittir.
  - Örnek: $a = 3$, $b = 4$, $p = 17$, $m = 6$, $k = 8 \Rightarrow (28 \bmod 17) \bmod 6 = 11 \bmod 6 = 5$.
- Yük faktörü: $\alpha = n/m$. Açık adreslemede $0 \leq \alpha < 1$.
- Doğrusal sınama (Knuth):
  - Başarılı arama: $S \approx \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$
  - Başarısız arama: $U \approx \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$
- Değer tablosu:

α	Başarılı (S)	Başarısız (U)
0,50	1,5	2,5
0,75	2,5	8,5
0,90	5,5	50,5

Detaylı Açıklamalar

Ayrık zincirleme

Her tablo gözü bir bağlı listenin başını (`head`) tutar. Gelen anahtarın adresi doluysa yeni düğüm o listeye eklenir.

Örnek ($m = 6$): 13 anahtarı $13 \bmod 6 = 1$ numaralı göze yerleşir. 1 anahtarı da 1 numaralı göze düşer. 19 geldiğinde $19 \bmod 6 = 1$ olduğu için yine 1 numaralı gözün listesine eklenir.

Derste tahtadaki çizimde 19 önce 3 numaralı göze çizildi. Bir öğrenci bunun 1 numaralı gözde olması gerektiğini söyledi. Hoca bu hatayı bilerek yaptığını, fark edilmesini beklediğini söyledi.

C ile ayrık zincirleme

Düğüm yapısı: anahtar, değer ve sonraki düğümün adresi tutulur.

```c
struct node {
    int key;
    char name[40];
    struct node *next;
};

struct hash {
    struct node *head;
    int count;
};

struct hash *hashTable = NULL;
```

Kod tahta karelerinden alındı.

`struct node` içinde kendi türüne işaretçi olduğu için `typedef` kısaltması yerine `struct node` kullanıldı.

`main` içinde tabloya yer açılır:

```c
hashTable = (struct hash *) calloc(M, sizeof(struct hash));
if (hashTable == NULL) {
    printf("Yer acilamadi");
    exit(0);
}
insertToHash(key, name);
searchInHash(key);
deleteFromHash(key);
```

Yeni düğüm oluşturma:

```
struct node *createNode(int key, char *name) {
    struct node *newNode = (struct node *) malloc(sizeof(struct node));
    newNode->key = key;
    strcpy(newNode->name, name);
    newNode->next = NULL;
    return newNode;
}
```

Ekleme (başa ekleme):

```
void insertToHash(int key, char name[]) {
    int hashIndex = key % M;
    struct node *newNode = createNode(key, name);
    if (!hashTable[hashIndex].head) {
        hashTable[hashIndex].head = newNode;
        hashTable[hashIndex].count = 1;
        return;
    }
    newNode->next = hashTable[hashIndex].head;
    hashTable[hashIndex].head = newNode;
    hashTable[hashIndex].count++;
}
```

Tahtadaki ekleme kodu `count` satırında yarım kaldı; son satır (`count++`) sözlü anlatıma göre tamamlandı. `M` global değişken olabilir ya da fonksiyona parametre olarak verilebilir.

Arama adımları:

1. `hashIndex = key % m` hesaplanır.
2. `hashTable[hashIndex].head`'den başlanır.
3. Düğüm NULL değilken ve anahtar eşleşmezken `next` ile ilerlenir.
4. Döngüden çıkınca bir bayrak (`flag`) ile anahtarın bulunup bulunmadığına bakılır. Bulunamadıysa "eleman bulunamadı" mesajı verilir.

Silme işlemi ödev olarak düşünülmeye bırakıldı.

Evrensel hashing

Tek bir sabit hash fonksiyonu yerine bir fonksiyon ailesi tanımlanır. Her çalıştırmada bu aileden bir fonksiyon rastgele seçilir.

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$$

a, b ve p rastgele seçilir. Tablo boyutu m sabittir.

Örnek: $a = 3$, $b = 4$, $p = 17$, $m = 6$, $k = 8$ için $(3 \cdot 8 + 4) \bmod 17 = 11$, $11 \bmod 6 = 5$.

Yük faktörü ve sınama sayıları

$\alpha = n/m$, tablonun doluluk oranıdır. Açık adreslemede α 0 ile 1 arasındadır, çünkü $m > n$'dir. Bu yüzden derste verilen tablo açık adresleme içindir.

Derste okunan ortalama sınama sayıları (doğrusal sınama, başarısız arama):

α	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9
Sınama	1,11	1,28	1,52	1,89	2,5	3,62	6,06	13	50,5

Başarısız aramada adım sayısı türetmesi:

- İlk bakılan gözün dolu olma olasılığı α, boş olma olasılığı $1 - \alpha$'dır.

- İlk iki gözün dolu olma olasılığı yaklaşık α^2'dir.
- Aramanın tam k. adımda bitmesi için ilk $k-1$ göz dolu, k. göz boş olmalıdır: $P(k) = \alpha^{k-1}(1-\alpha)$.
- Ortalama adım sayısı: $\sum_{k\geq 1} k\,\alpha^{k-1}(1-\alpha) = \frac{1}{1-\alpha}$ (geometrik seri).

Başarılı ve başarısız arama farkı: Başarılı aramada anahtar bulununca durulur. Başarısız aramada boş göz görülene kadar devam edilir. Tablo dolunca öbekler birleşir ve boş göz azalır. Bu yüzden %90 dolulukta başarılı arama 5,5, başarısız arama 50,5 sınama sürer.

Ayrık zincirlemede arama

Ayrık zincirlemede α 1'den büyük olabilir.

- Başarılı arama: $1 + \alpha/2$ (1, hash hesabı; $\alpha/2$, listenin ortalama yarısı)
- Başarısız arama: $1 + \alpha$ (bütün liste gezilir)

Örnek: $n = 100$, $m = 25 \Rightarrow \alpha = 4$. Başarısız aramada hash hesabından sonra ortalama 4 düğüm gezilir. Başarılı arama ortalama $1 + 4/2 = 3$ adımdır.

Teoride 100 elemanın hepsi aynı göze düşebilir. Pratikte iyi bir hash fonksiyonuyla her listede ortalama α eleman bulunur.

Doğrusal ve ikinci dereceden sınama karşılaştırması

Tahtada başarılı arama için iki eğri çizildi: yatay eksende α (0,2-1), dikey eksende ortalama sınama sayısı. Eğriler 1'e kadar sınırlı olduğu için ikisi de açık adresleme yöntemidir (ayrık zincirleme değil). Daha hızlı yükselen eğri doğrusal sınama, diğeri ikinci dereceden sınamadır. Aynı doluluktaki iki tabloda ikinci dereceden sınama daha az sınama ile arar.

Ders sonrası notlar

- **Başarısız arama türetmesinin sonucu:** Derste geometrik seri açılımından "$1-\alpha$" çıktığı söylendi. Doğru sonuç $\frac{1}{1-\alpha}$'dır. Örneğin $\alpha = 0{,}9$ için ortalama 10 sınama bulunur. Bu türetme her sınamanın bağımsız olduğu varsayımına (uniform hashing) dayanır. Doğrusal sınamada öbeklenme olduğu için gerçek değer daha büyüktür: $\frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$, yani 50,5.
- **Tablodaki değerlerin anlamı:** Derste okunan 1,11 / 1,28 / ... / 50,5 değerleri doğrusal sınamada başarısız arama formülünden gelir. 0,7 değeri derste atlandı: 6,06. İlk değer yuvarlanınca 1,12'dir; derste 1,11 diye okundu.
- **"Pratikte $O(1)$" ifadesi:** Hoca "500 elemanda 50 adımda, 5000 elemanda 90 adımda ulaşıyorsanız bu da $O(1)$'dir" dedi. Adım sayısı n ile büyüyorsa $O(1)$ değildir. Doğru ifade şudur: $\alpha = n/m$ sabit tutulursa (tablo büyüdükçe m de büyütülürse) ortalama arama $\Theta(1+\alpha) = O(1)$'dir.
- **Evrensel hashingin amacı:** Derste evrensel hashingin amacı güvenlik ve "parolanın ağ üzerinden tek yönlü şifrelenerek taşınması" olarak anlatıldı. Bu iki ayrı konunun karışmasıdır.
  - Evrensel hashing, fonksiyonu rastgele seçerek her girdi için çakışma olasılığını ortalamada en fazla $1/m$'de tutar. Böylece kötü niyetli biri çakışma üreten girdiler hazırlayamaz. Amaç hash tablosunun ortalama performansını garanti etmektir.
  - Parolalar SHA ailesi gibi kriptografik hash fonksiyonlarıyla (ve tuz/salt ile) saklanır. Bu fonksiyonlar evrensel hashing ailesinden değildir.
  - Evrensel hashingde p asal ve anahtar uzayından büyük, $a \in \{1, \ldots, p-1\}$, $b \in \{0, \ldots, p-1\}$ seçilir.
- **Ayrık zincirleme sayısal örneği:** Derste $n = 100$, $m = 25$ için başarılı arama "4 bölü 2'den 2, 3" diye karışık söylendi. Doğrusu $1 + \alpha/2 = 3$'tür.
- $\alpha = 3$ **örneği:** Hoca başarılı arama için "$\alpha = 3$ ise 3 bölü 2'den 1,5" dedi. 1,5 yalnız liste kısmıdır; formüldeki 1 de eklenince başarılı arama $1 + 1{,}5 = 2{,}5$ adım olur.
- **Bellek ayırma:** Tabloya yer açarken `calloc` kullanılması doğru bir seçim: bütün `head` alanlarını NULL, `count` alanlarını 0 yapar. `malloc` kullanılsaydı `head` alanları ayrıca NULL yapılmalıydı, yoksa `if (!head)` kontrolü çöp değer okur.

- **Tahtadaki küçük kod hataları:** `createNode` içinde büyük olasılıkla `size(struct node)` yazılmış; doğrusu `sizeof(struct node)`. Yer açılamadığında `exit(0)` yerine hata kodu (`exit(1)` ya da `EXIT_FAILURE`) döndürmek daha doğrudur. `strcpy` 40 karakterden uzun isimde taşma yapar; `strncpy` ya da uzunluk kontrolü kullanılmalıdır.
- Altyazıda “Spread Chaining”, “LinkedIn liste” diye geçen terimlerin doğruları: separate chaining, linked list.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.